Gallery Guide Studio

Project summary for architecture collaboration — prepared so a second Claude session (or a person) can get the full picture in one read before reasoning about how this fits into a larger system.

What This Is

Gallery Guide Studio is a prototyping tool for building and testing branching, voice-capable conversation flows — the kind of scripted-but-flexible agent you'd put at a museum front desk, a clinic check-in kiosk, or any “talk to a kiosk” scenario. A non-engineer designs the flow visually as a set of connected cards on a FigJam board, an LLM turns each card's brief into actual step-by-step dialogue, and the resulting flow can be tested immediately as a real text/voice conversation — in the browser, or on a phone.

It is a **local, single-user prototyping tool**, not a deployed product — see “Architectural Constraints” below before assuming otherwise. That's the single most important fact for reasoning about it as a component of something bigger.

The Core Idea: A Deterministic Conversation Engine

The central design decision, and the thing most worth understanding first: a conversation's logic is **not** re-decided by an LLM on every turn. Each stage of the flow (a “playbook”) is an ordered table of steps, and a plain mechanical function walks that table forward one request at a time. This trades some flexibility for reliability — the flow can't wander off-script, stall, or forget where it is, because “where it is” is a hard piece of state (a box id + step index), not something re-inferred from conversation history each turn.

- **BOT** — the agent speaks a line (an LLM phrases it naturally from a fixed instruction, but never decides *whether* to say it).
- **USER** — wait for the visitor's reply; optionally extract and save it to a named variable (e.g. their name).
- **NAV** — jump to another playbook. A single NAV at a point is an unconditional jump; two or more in a row is a branch, classified by an LLM call scoped to just “which of these N labeled conditions best matches what they just said” — never an open-ended “what should happen next” judgment call.

Variables captured from USER steps persist for the rest of the session and are available to every later BOT line. The engine lives in `src/app/api/chat/route.ts` (the `walk()` function).

How a Flow Gets Authored

- **Design in FigJam** — each playbook is a card: a title (“PLAYBOOK <code>: <label>”), a freeform prose brief, and a step table (initially empty).
- **Generate the steps** — a FigJam plugin (installed in dev mode, in `figjam-plugin/`) reads each card's brief and calls Anthropic directly to fill in the step table, keeping stacked cards spaced consistently as they grow. It can be triggered by hand inside Figma, or remotely from the app's “Generate Steps in FigJam” button via a small polling job queue (necessary because Figma's write API is plugin-only — the REST API is read-only).
- **Bring it into the app** — the student exports the finished board as a PNG and uploads it. Computer vision finds each blue “open playbook” button's position; if a FigJam file is connected, each detected button is also matched to its real synced content by label, so the app ends up with an interactive, clickable image of the actual

board.

- **Edit content** — clicking a button opens a draft editor (prompt + step table) with explicit Save/Cancel; nothing commits until Save. A symmetrical “Update FigJam file” action can write local edits back into the live board through the same plugin relay.

How a Conversation Runs

The chat UI sends the full playbook table plus current position/variables on every request; the server walks forward exactly one turn's worth (one or more steps, per the rules above) and returns the resulting text plus the new position. Text replies are optionally spoken aloud via ElevenLabs TTS. Voice input has two modes: a manual push-to-talk mic (click, talk, click Send), and an experimental “hands-free” mode (continuous listening, auto-sends after a configurable pause, mutes itself while the bot is speaking).

> **Known issue, unresolved:** hands-free voice mode is not yet reliable on iOS Safari. Several real bugs have been found and fixed (an iOS audio-autoplay restriction, a race causing duplicate sends, a state bug that could permanently disable the mic after one failed request), but a further issue remains where iOS's speech recognition can silently stop responding while still reporting itself as active. A watchdog/auto-restart mitigation is in place but not yet confirmed to fully resolve it. Manual push-to-talk works reliably everywhere.

Testing on a Phone

Because the app itself is desktop-oriented, a separate minimal page (`/phone`) provides a chat-only view for testing on an actual phone. A “Phone testing” button encodes the current flow (playbooks, voice settings — deliberately excluding API keys) into a URL and shows it as a link and QR code; the phone enters its own API keys once, stored only in that browser. This is a one-time snapshot, not a live sync. Since the dev server only listens on `localhost`, reaching it from a phone requires a tunnel (currently Cloudflare Tunnel) — there is no deployed version of this app anywhere.

Tech Stack

Layer	Choice
Framework	Next.js 16 (App Router, Turbopack), TypeScript, React 19
Styling	Tailwind CSS v4
LLM	Anthropic via the Vercel AI SDK (`ai`, `@ai-sdk/anthropic`) — Claude Sonnet 5 for natural-language phrasing, Haiku 4.5 for short structured extraction/classification calls
Text-to-speech	ElevenLabs (REST, proxied through `/api/tts`)
Design surface	Figma REST API (read) + Figma Plugin API (write, via the local plugin)
Voice input	Browser Web Speech API (`SpeechRecognition`)
QR codes	`qrcode` (client-side, no external service)
State/persistence	Browser `localStorage` only — no database

Layer	Choice
Validation	Zod

API Surface (current Next.js API routes)

Route	Purpose
`POST /api/chat`	The conversation engine — walks the active playbook forward one turn, streams the reply
`POST /api/generate-playbooks`	LLM-generates step tables for one or more playbooks from their briefs
`POST /api/sync-figma`	Reads real playbook content/connections from a Figma board via the REST API
`POST/GET /api/figma-jobs`	Job queue the FigJam plugin polls — remote-triggers “Generate Steps” / “Update FigJam file”. In-memory only, single shared queue, one app + one plugin session assumed
`GET/POST /api/figma-jobs/[id]`	Poll/update one job's status
`GET /api/tts`	Proxies ElevenLabs text-to-speech
`POST /api/detect-labels`	Vision call reading button labels off an uploaded board PNG

Data Model (essentials)

Everything lives in one client-side state object, persisted to `localStorage`:

```
StudioState {
boxes: PlaybookBox[] // the playbooks/stages
activeBoxId, activeStepIndex // where the conversation is right now
variables: Record<string,str> // values captured from USER steps
masterPlaybook: string // global persona/tone/rules, all stages
voiceId, voiceSettings, voicePresets
anthropicApiKey, elevenLabsApiKey, figmaApiKey // typed in by the user
figmaFileUrl, imageDataUrl, imagePixelSize, figmaWriteDirty
}

PlaybookBox {
id, label, code // code parsed from "PLAYBOOK <code>: label"
instructionPrompt: string // the brief that generates steps
steps: PlaybookStep[]
x, y, width, height // position on the canvas image, 0-100%
leadsTo?: string[] // real neighbor ids, from Figma connectors
figmaSourceId?: string // Figma TABLE node id, if synced
}

PlaybookStep {
id, option: 'BOT'|'USER'|'NAV', instruction: string
```

```
navTarget?: string // NAV only — target box id
variable?: string // USER only — name to save the answer under
}
```

Architectural Constraints — Read Before Designing Around This

> This is a single-user, local-machine prototyping tool. Every point below is a real, current limitation, not a stylistic choice — any "component of a bigger system" design needs to either work within these or plan to change them.

- **No backend database.** All state is a single JSON blob in one browser's `localStorage`. Nothing is shared across devices or users except by manually exporting a snapshot (the phone-link mechanism).
- **No authentication or multi-tenancy.** There is exactly one "studio" per browser profile. No concept of accounts, ownership, or permissions.
- **Not deployed anywhere.** Runs only via `npm run dev` on the developer's own machine. Phone access requires a manually-started tunnel (Cloudflare Tunnel currently) with a URL that changes every restart.
- **The FigJam write path is hardcoded to localhost.** The plugin-to-app job queue only ever talks to `http://localhost:3000` — "Generate Steps" and "Update FigJam file" would need real rework to function against a hosted version of this app.
- **API keys are end-user-supplied and client-side.** Anthropic, ElevenLabs, and Figma credentials are typed into the app by whoever's using it and stored in that browser only — there is no server-managed key vault or usage metering.
- **No automated tests.** Correctness has been verified through manual/live testing each session, not a test suite.
- **Dev-mode only where it matters most.** The Next.js dev server (not a production build) is what's actually been run and tunneled throughout — a production build (`next build && next start`) has not been evaluated for this workflow yet.

Where to Find Things

Repository: `github.com/GPonGithub/gallery-guide-studio`, branch `v3-step-engine`. Three markdown files in the repo root carry the detailed, chronological build history if deeper context is ever needed: `HANDOFF.md` (the original architecture rationale), `SESSION_HANDOFF.md`, and `FIGJAM_PLUGIN_HANDOFF.md` (the most detailed and current of the three, including the exact state the hands-free voice debugging was paused in).

Open Questions Worth Discussing

- What's the actual integration boundary — does the bigger system call this app's conversation engine as a service, embed its UI, or just reuse the step-table concept as a library/spec?
- Does the bigger system need multiple simultaneous users/flows? If so, the localStorage-only state model needs to become a real backend before anything else here matters much.
- Does authoring stay FigJam-based, or would the bigger system want its own flow-authoring surface (in which case the FigJam plugin/sync layer may be scoped out entirely)?

- Is iOS voice reliability a hard requirement for the target use case? If hands-free voice is central to the bigger system's UX, that's worth resolving or redesigning before building on top of it.
- Does the bigger system need centrally-managed API keys/usage limits, rather than each user supplying their own?

Prepared from a working session on Gallery Guide Studio. Happy to answer follow-up questions directly, or point to specific files/commits for anything this summary compresses too far.